

# Sécurité des microservices

## Objectifs :

1. Installer Keycloak comme docker
2. Création d'un client
3. Sécuriser le microservice Gateway
4. Authorisations selon les rôles

## Installer Keycloak comme docker

<https://www.keycloak.org/getting-started/getting-started-docker>

```
docker run -d -p 8090:8080 -e KC_BOOTSTRAP_ADMIN_USERNAME=admin  
-e KC_BOOTSTRAP_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak:26.3.4 start-dev
```

## Création d'un client

|               |                        |
|---------------|------------------------|
| Client type ? | OpenID Connect         |
| Client ID * ? | dept-app               |
| Name ?        | dept-app               |
| Description ? | Department application |

**Client authentication**  On

**Authorization**  Off

**Authentication flow**

☐ Standard flow
 ☐ Direct access grants

☐ Implicit flow
 ☒ Service accounts roles

☐ Standard Token Exchange

☐ OAuth 2.0 Device Authorization Grant

☐ OIDC CIBA Grant

**Root URL**

**Home URL**

---

**Client Secret**




Obtenir un « Access Token » en utilisant les « Client credentials »

<http://localhost:8090/realms/master/protocol/openid-connect/token>

## POSTMAN

**POST** <http://localhost:8090/realms/master/protocol/openid-connect/token>

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings

☐ none
 ☐ form-data
 ☒ x-www-form-urlencoded
 ☐ raw
 ☐ binary

| Key                                               | Value                            |
|---------------------------------------------------|----------------------------------|
| <input checked="" type="checkbox"/> grant_type    | client_credentials               |
| <input checked="" type="checkbox"/> client_id     | dept-app                         |
| <input checked="" type="checkbox"/> client_secret | Gpp1MTriwwcymbF8XhMSKrPLn6qeqx3G |
| <input checked="" type="checkbox"/> scope         | openid email profile             |

# Sécuriser le microservice Gateway

- Ajouter les dépendances suivantes :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-oauth2-resource-server</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-oauth2-jose</artifactId>
</dependency>
```

- Créer la classe SecurityConfig dans le package security

```
@Configuration
@EnableWebFluxSecurity
public class SecurityConfig
{

    @Bean
    public SecurityWebFilterChain springSecurityFilterChain(ServerHttpSecurity
serverHttpSecurity) {
        serverHttpSecurity.authorizeExchange(exchanges -> exchanges.pathMatch-
ers(HttpMethod.GET).permitAll()
            .pathMatchers("/api/departments/**").authenticated()
            .pathMatchers("/api/teachers/**").authenticated())
            .oauth2ResourceServer(oAuth2ResourceServerSpec ->
oAuth2ResourceServerSpec
                .jwt(Customizer.withDefaults()));

        serverHttpSecurity.csrf(csrfSpec -> csrfSpec.disable());
        return serverHttpSecurity.build();
    }
}
```

Ajouter la configuration de la sécurité au fichier application.yml

```
security:
  oauth2:
    resourceserver:
      jwt:
        jwk-set-uri: "http://localhost:8090/realms/master/protocol/openid-connect/certs"
```

Tester l'api :

<http://localhost:8888/api/departments/IT>

GET http://localhost:8888/api/departments/IT

Params Authorization Headers (7) Body Scripts Tests Settings

Query Params

| Key |
|-----|
|-----|

Body Cookies Headers (10) Test Results

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "depName": "Info Tech",
4   "depCode": "IT"
5 }
```

Modifier permitAll() en authenticated()

```
exchanges.pathMatchers(HttpMethod.GET).authenticated()
```

Tester l'api :

<http://localhost:8888/api/departments/IT>

GET http://localhost:8888/api/departments/IT

Params Authorization Headers (7) Body Scripts Tests Settings

Query Params

| Key | Value | Description |
|-----|-------|-------------|
|-----|-------|-------------|

Body Cookies Headers (9) Test Results

401 Unauthorized

## Utiliser “Authorization”

GET

http://localhost:8888/api/departments/IT

Params

Authorization

Headers (8)

Body

Scripts

Tests

Settings

Auth Type

OAuth 2.0

The authorization data will be automatically generated when you send the request. Learn more about [OAuth 2.0](#) authorization.

Add authorization data to

Request Headers

Grant type

Client Credentials

Access Token URL ⓘ

http://localhost:8090/realms/master/protoc...

Client ID ⓘ

dept-app

Client Secret ⓘ

.....

Scope ⓘ

openid email profile

Client Authentication ⓘ

Send client credentials in body

Advanced

ⓘ You can add more specific customizations to your OAuth2 requests here. Learn more about [configuration](#) ↗

Body

Cookies

Headers (10)

Test Results

{ } JSON

Preview

Visualize

1 {

2 |

3 | "id": 1,

4 | "depName": "Info Tech",

5 | "depCode": "IT"

6 | }

## Authorisations selon les rôles

```
.pathMatchers("/api/departments/**").hasRole("ADMIN")
```

- Créer une real role 'ADMIN'
- Affecter ADMIN au client dept-app

dept-app OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings Keys Credentials Roles Client scopes **Service accounts roles** Sessions Adva

**i** To manage detail and group mappings, click on the username [service-account-dept-app](#)

→ ☒ Hide inherited roles Assign role ▼ Unassign Refresh

| <input type="checkbox"/> Name                 | Inherited |
|-----------------------------------------------|-----------|
| <input type="checkbox"/> ADMIN                | False     |
| <input type="checkbox"/> default-roles-master | False     |

- Vérifier l'existence du rôle ADMIN sur [jwt.io](#)

## Créer la classe KeycloakRoleConverter

```
package com.nadhem.gatewayserver.security;

import org.springframework.core.convert.converter.Converter;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.oauth2.jwt.Jwt;
import java.util.ArrayList;
import java.util.Collection;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class KeycloakRoleConverter
implements Converter<Jwt, Collection<GrantedAuthority>> {

    @Override
    public Collection<GrantedAuthority> convert(Jwt source) {
        Map<String, Object> realmAccess = (Map<String, Object>)
source.getClaims().get("realm_access");
        if (realmAccess == null || realmAccess.isEmpty()) {
            return new ArrayList<>();
        }
    }
}
```

```

    }
    Collection<GrantedAuthority> returnValue = ((List<String>) realmAccess.get("roles"))
        .stream().map(roleName -> "ROLE_" + roleName)
        .map(SimpleGrantedAuthority::new)
        .collect(Collectors.toList());
    return returnValue;
}
}

```

## Modifier la classe SecurityConfig

```

package com.nadhem.gatewayserver.security;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.convert.converter.Converter;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.AbstractAuthentica-
    tionToken;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.reactive.Enable-
    WebFluxSecurity;
import org.springframework.security.config.web.server.ServerHttpSecurity;
import org.springframework.security.oauth2.jwt.Jwt;
import org.springframework.security.oauth2.server.resource.authentica-
    tion.JwtAuthenticationConverter;
import org.springframework.security.web.server.SecurityWebFilterChain;
import reactor.core.publisher.Mono;
import org.springframework.security.oauth2.server.resource.authentica-
    tion.ReactiveJwtAuthenticationConverterAdapter;

@Configuration
@EnableWebFluxSecurity
public class SecurityConfig {

    @Bean
    public SecurityWebFilterChain springSecurityFilterChain(ServerHttpSecu-
    rity serverHttpSecurity) {
        serverHttpSecurity.authorizeExchange(exchanges -> //exchanges.path-
    Matchers(HttpMethod.GET).permitAll()
        exchanges.pathMatchers("/api/depart-
    ments/**").hasRole("ADMIN")
        .pathMatchers("/api/teachers/**").authenticated())
        .oauth2ResourceServer(oAuth2ResourceServerSpec ->
    oAuth2ResourceServerSpec
        // .jwt(Customizer.withDefaults()))
        .jwt(jwtSpec -> jwtSpec.jwtAuthenticationCon-
    verter(grantedAuthoritiesExtractor())));

        serverHttpSecurity.csrf(csrfSpec -> csrfSpec.disable());
        return serverHttpSecurity.build();
    }
}

```

```
private Converter<Jwt, Mono<AbstractAuthenticationToken>>
grantedAuthoritiesExtractor() {
    JwtAuthenticationConverter jwtAuthenticationConverter =
        new JwtAuthenticationConverter();
    jwtAuthenticationConverter.setJwtGrantedAuthoritiesConverter
        (new KeycloakRoleConverter());
    return new ReactiveJwtAuthenticationConverterAdapter(jwtAuthentica-
tionConverter);
}
```

Enlever le rôle ADMIN du client dept-app puis Tester avec POSTMAN l'accès à l'api

<http://localhost:8888/api/departments/IT>